

Systematic Mutation and Black-Box Probing for SQL Injection WAF Bypass: A Live Testbed and Family-Level Analysis

Tran Xuan Quyen

*School of Info. & Comm. Technology
HUST*

Hanoi, Vietnam

quyen.tx252593m@sis.hust.edu.vn

Co Huy Dung

*School of Info. & Comm. Technology
HUST*

Hanoi, Vietnam

dung.ch252592m@sis.hust.edu.vn

Nguyen Duc Toan

*School of Info. & Comm. Technology
HUST*

Hanoi, Vietnam

toan.nguyenduc1@hust.edu.vn

Abstract—Attackers routinely evade Web Application Firewalls (WAFs) by mutating SQL injection (SQLi) payloads. Prior studies report high bypass rates, yet these aggregate figures often conflate heterogeneous operators and depend on search biases. Consequently, defenders need reproducible, family-resolved measurements tied to explicit labels.

This paper presents an offensive methodology that generates discrete SQLi variants through dialect seeds, deployment-specific augmentations, and learned transform rules. We issued black-box HTTP probes against a live AWS WAF-fronted application, collecting per-attempt JSONL logs. In our archived run of 5,000 variants, our operational classifier identified a 41.7% bypass rate. However, family-level analysis reveals extreme heterogeneity: identity, case-mix, and null-byte cohorts achieved bypass rates near 99%, while encoding and hex-style cohorts were entirely blocked. This indicates that policy stress concentrates in specific syntactic strategies.

As a secondary control, we applied repeated RFC percent-decoding and scored strings with a TF-IDF logistic-regression surrogate. On a 20-request pilot, accuracy remained at 42.9% for both raw and canonicalized inputs, suggesting that naive decoding neither closes the live bypass gap nor rescues a simple offline model. All reported rates are conditional on the specific experimental configuration; the core contribution lies in the explicit methodology, decomposed statistics, and reproducible artifact.

Index Terms—SQL injection, web application firewall, black-box attack, adversarial mutation, bypass measurement, variant families

I. INTRODUCTION

Web applications remain the primary interface for online services, with Web Application Firewalls (WAFs) widely deployed to block injection attacks [9], [10]. From an attacker’s perspective, the key challenge is navigating a specific deployment’s concrete rules and parsers to achieve a security-relevant application outcome. Signatures can be evaded by semantic-preserving rewrites, and parsers can disagree on decoding and parameter binding [11]–[13]. While these phenomena are well known, measuring the conditions and specific mutation structures under which bypasses occur remains a scientific challenge.

This paper contributes an operational, attack-centric evaluation. We systematically mutate SQLi fragments, probe

a production-style AWS WAF deployment, and analyze outcomes at both aggregate and variant-family levels. We release our implementation as a reproducible artifact (`paper/experiment`) to provide red and blue teams with empirical distributions for prioritization and regression testing.

A. Problem Statement

SQLi evasion combines representation-level mutation—altering substrings or encodings seen by a pattern matcher—and interpretation-level divergence, where the WAF and application decode the same sequence differently [2].

Prior adversarial methods demonstrate that the attack space is combinatorial. Systems like SSQli [1], GSQli [7], and WAF-A-MoLE [6] achieve high bypass rates by searching this space. However, success is rarely uniform across operators. Published rates seldom decompose by operator family on live managed WAFs, obscuring which mutations survive. Furthermore, while mutating HTTP structure induces parsing discrepancies [2], we focus on URL-visible injection with deterministic seed remapping. This minimizes parameter-name mismatch, providing a controlled setting to study representation-level evasion against a fixed policy.

B. Objectives and Research Questions

We structure our empirical evaluation around three core questions:

- **RQ1 (Scale and semantics).** Under an explicit bypass definition, what fraction of attempts succeed against our AWS WAF lab, and how does this relate to unique payload cardinality?
- **RQ2 (Structure).** How much bypass mass concentrates in specific mutation families, and which are systematically suppressed?
- **RQ3 (Normalization control).** Does minimal RFC percent-decoding materially improve a simple offline surrogate’s ability to classify strings?

In addressing these questions, we formalize a black-box attack pipeline for managed WAF assessment. We provide an empirical characterization of a live AWS WAF run, detailing

Layers: lexical SQLi mutations; URL/query encodings; edge WAF normalization; application parser/ORM; observable HTTP/app signals

Fig. 1. Conceptual layering for SQLi WAF bypass experiments.

unique payload counts and family-level bypass rates that reveal sharp heterogeneity. Finally, we include a secondary control showing that basic canonicalization does not immediately rescue simple classifiers.

C. Paper Organization

Section II covers related work. Section III details our methodology. Section IV presents the evaluation answering RQ1–RQ3. Section V concludes the paper.

II. BACKGROUND AND RELATED WORK

Offensive SQLi evaluation bridges adversarial search against detectors, HTTP semantics, and operational security measurement. We highlight strands that motivate our focus on family-resolved reporting.

A. Mutation-Based SQLi Attacks

Adversarial methods search for inputs that preserve malicious intent while evading detectors. Reinforcement learning approaches like SSQli [1], GAN-based methods like GSQli [7], and semantics-preserving fuzzing like WAF-A-MoLE [6] achieve high bypass rates. The underlying attack space is combinatorial, involving comment insertions, encodings, and dialect-specific spellings that interact non-monotonically with tokenized defenses [14]. Consequently, the relevant metric is not just the aggregate bypass rate, but which specific compositions survive, as this informs both attacker and defender strategies.

B. Parsing Discrepancies and Defense Evaluation

When components disagree on decoding order or structural boundaries, attackers can satisfy edge predicates while delivering malicious semantics [2]. WAFFLED demonstrated this across major WAFs. Our experiments stabilize parameter binding via remapped seeds to isolate representation-level variance.

Hybrid defenses aim to raise evasion costs through better data [8], shadow models [5], or preprocessing [4]. However, evaluating these defenses requires paired offensive benchmarks. Without logs tying attempts to mutation families, it is difficult to determine if a mitigation removes high-yield operators or merely reshuffles the search order. Our work supplies a log-grounded baseline for such comparisons.

C. Metrics and Confounding Threats

Black-box evaluation relies on observable HTTP outcomes. Since different probes map the same vulnerability to different observables, any operational definition of a bypass embeds epistemic commitments. Transparent scripting and published logs mitigate this issue, though they do not eliminate it entirely.

Seeds → augmentations & rules → 2nd-layer transforms → probes → logs

Fig. 2. Offensive evaluation workflow; policies configurable in config.json.

III. METHODOLOGY

This section details our adversarial evaluation methodology, including the experimental environment, adversary capabilities, and the attack pipeline. Artifacts are available under paper/experiment.

A. Experimental Environment

We deployed the study on AWS, sending traffic through an AWS WAF Web ACL in us-east-1 before reaching an EC2-hosted application. This mirrors a production environment where the backend only sees what the WAF forwards.

The Web ACL (test-waf) features three sequential rules:

- 1) **Block-SQLi-Strict-Pattern:** Targets simple tautologies (e.g., 1=1) evaluated across all query arguments after decoding and normalization.
- 2) **AWS-AWSManagedRulesSQLiRuleSet:** Amazon’s managed SQLi bundle, set to block any hit, providing broad vendor coverage across the request structure.
- 3) **Final-Optimized-SQLi-Rule:** A custom regular expression stage targeting specific lab leftovers, such as hex-flavored encodings and null-byte obfuscation.

The endpoint is a custom web application on a t3.micro EC2 instance. We drove probes manually via scripts without relying on generic scanners like sqlmap. Outcomes were observed directly from HTTP responses: a 403 indicated a block, while a 200 indicated the request reached the application.

B. Adversary Model and Attack Pipeline

We model a network adversary capable of sending HTTP/1.1 requests, observing responses, and adapting mutations based on logs. The adversary lacks white-box access, aligning with typical red-team constraints [1], [6].

The end-to-end pipeline operates as follows:

$$\mathcal{M} : \text{Seeds} \rightarrow \text{Augmentations} \rightarrow \text{Transforms} \rightarrow \mathcal{V}, \quad (1)$$

$$\mathcal{V} \xrightarrow{\text{HTTP}} \text{WAF} \rightarrow \text{App} \rightarrow \text{Label}.$$

The driver serializes each variant in $\mathcal{V}$ as a GET request, logs the response, and outputs a JSON object.

C. Staged Mutation Generation

To balance diversity and reproducibility, generation is staged:

- **Seeds:** We use PostgreSQL- and Oracle-oriented templates, affecting syntax and tokenization. Numeric seed tokens are remapped to stable string prefixes to align WAF and application parameter tracking.
- **Learned Rules:** The harness injects deployment-specific variants, biasing the search towards historically informative outcomes.

- **Transforms:** An optional second layer applies further evasion transforms subject to configuration limits. Consequently, the empirical distribution reflects practical search limits rather than a uniform draw from all possible SQLi strings.

D. Outcome Semantics and Metrics

Each attempt i yields an HTTP status s_i and body b_i , and is labeled $y_i \in \{\text{bypass}, \text{blocked}, \text{other}\}$. A **blocked** label denotes edge denial. A **bypass** label indicates a WAF-pass with application-visible signals consistent with effective SQLi side effects. **Other** covers ambiguous outcomes.

Let N be the total attempts and N_{bp} the bypass count. The aggregate bypass rate is:

$$\text{BR}_{\text{agg}} = \frac{N_{\text{bp}}}{N} \quad (2)$$

For a mutation family f , the family-level bypass rate is $\text{BR}_f = N_{f,\text{bp}}/N_f$. We also measure unique payload cardinality to identify whether high attempt-level rates result from diverse payloads or string reuse.

E. Secondary Control

To assess normalization effects, we applied fixpoint unquote_plus decoding to probe strings and trained a TF-IDF logistic-regression surrogate (ml_baseline.py) on 20 labeled HTTP templates. This pilot tests whether minimal RFC-relevant decoding aligns feature spaces sufficiently to improve a simple classifier.

IV. EXPERIMENTS AND RESULTS

We answered RQ1–RQ3 using data exported from live AWS WAF probing and an RFC decoding surrogate pilot. Scripts were executed in a Windows environment using standard data science libraries (scikit-learn, joblib).

A. Experimental Flow

We followed a consistent workflow:

- 1) Start with SQLi seeds and remap IDs to the target format.
- 2) Apply mutation rules and send GET requests to the endpoint.
- 3) Classify responses as **bypass**, **blocked**, or **other**.
- 4) Aggregate outcomes to compare family-level behavior.

We focus on attempt-level counts (RQ1) from `aws_probe_summary.json`, family-level rates (RQ2) from the trailing slice of `aws_probe_results.jsonl`, and surrogate pilot accuracy (RQ3) from `defense_eval.json`.

B. RQ1: Scale, Labels, and Unique Payloads

The archived run reports $N = 5,000$ variants tested, with $N_{\text{bp}} = 2,083$ bypasses, resulting in $\text{BR}_{\text{agg}} = 41.7\%$ (Table I). Among the trailing 5,000 requests, we observed only 19 distinct payload strings in the bypass set and 34 in the blocked set, with no overlap. This indicates that the search reused similar surface forms under different transform labels. Our workload targeted URL-visible slots with percent-encoding and dialect modifications, highlighting the necessity of extra probes for full-body inspection.

TABLE I
AWS WAF LAB ATTACK CORPUS (EXPORTED AGGREGATE).

Metric	Value	Notes
Variants tested N	5,000	<code>summary.json</code>
Bypass count N_{bp}	2,083	$\text{BR}_{\text{agg}} = 41.7\%$
Blocked count	2,793	Same summary
Unique bypass payloads	19	Last-run JSONL
Unique blocked payloads	34	Last-run JSONL

C. RQ2: Family-Level Heterogeneity

Bypass success is highly concentrated. Figure 3 and Table II show that the *spawn_identity*, *spawn_case_mix*, and *spawn_null_byte* cohorts achieved bypass rates near 100%. Conversely, *spawn_encoding* and *spawn_hex_style* were fully blocked ($\text{BR}_f = 0$). This pattern demonstrates that the WAF policy effectively handles nested encodings but struggles with specific structural mutations.

$\text{BR}_{\text{agg}} \approx 42\%$ ($N=5,000$); encoding/hex blocked;
identity/case/null-byte $\text{BR}_f \approx 99\%$

Fig. 3. Contrasting family-level bypass behavior.

TABLE II
SELECTED MUTATION FAMILIES: ATTEMPTS N_f AND BR_f .

Family	N_f	BR_f (%)	Role
<i>spawn_identity</i>	973	≈ 98.7	High yield
<i>spawn_null_byte</i>	972	≈ 98.8	High yield
<i>spawn_case_mix</i>	142	≈ 100.0	High yield
<i>spawn_encoding</i>	1,940	0	Fully blocked
<i>spawn_hex_style</i>	973	0	Fully blocked

D. RQ3: Surrogate Accuracy and RFC Decoding

We evaluated a TF-IDF logistic-regression surrogate on a 20-template HTTP pilot corpus (Table III). Test accuracy remained at 42.9% for both raw and `unquote_chain`-canonicalized strings. In this small sample, fixpoint percent-decoding neither improved nor degraded surrogate performance.

TABLE III
OFFLINE SURROGATE PILOT TEST ACCURACY ($n=20$).

Condition	Test accuracy	Notes
Raw strings	0.429	Same split
After unquote_chain	0.429	Fixpoint decode

E. Threats to Validity and Reproducibility

Threats. Our labels rely on script-specific heuristics. Results are bound to a specific policy snapshot and single-channel URL injection. Furthermore, the search distribution is explicitly non-uniform, reflecting operational biases. **Reproducibility.** Summaries can be regenerated with scripts in the repository. Executing `run_probe.py` yields fresh logs, though counts will naturally drift as WAF policies evolve. Probing was conducted exclusively against authorized targets.

V. DISCUSSION AND CONCLUSION

This work decomposes SQLi WAF bypass analysis along semantics, scale, and structure under an induced search distribution.

A. Research Question Answers

RQ1. The archived run ($N = 5,000$) yields a 41.7% bypass rate but only 19 unique bypass payloads. High attempt-level rates coexist with concentrated string reuse.

RQ2. Bypass probability varies drastically across families. Identity-, case-, and null-byte-oriented spawns achieve nearly 100% success, while encoding and hex-style spawns are fully blocked. This highlights that global metrics mask structural holes.

RQ3. Fixpoint percent-decoding did not improve our offline surrogate pilot’s accuracy, demonstrating that live bypass vulnerabilities cannot be trivially resolved by simple canonicalization. Stronger mitigations require richer feature interactions and protocol-complete normalization.

B. Implications and Conclusion

For defenders, policy tuning and regression testing must be family-aware; relying on aggregate dashboards is insufficient. Edge decisions should be combined with robust application-layer controls. For offensive researchers, publishing induced search configurations and explicit outcome logs is vital for reproducible science.

We presented a reproducible black-box methodology for SQLi WAF evaluation. The core takeaway is that meaningful offensive evaluation demands decomposed statistics and transparent search bias. Our family-level analysis identifies specific operational vulnerabilities, reinforcing the need for layered, context-aware defenses.

Limitations and Ethics. Our findings are bounded by a single policy snapshot and URL injection channel. All methodology is strictly intended for authorized testing environments.

ACKNOWLEDGMENT

The authors thank the faculty and staff of the School of Information and Communication Technology, Hanoi University of Science and Technology, for their academic guidance and support throughout this research. The authors also appreciate the constructive feedback from colleagues and peers, which helped improve the quality of this work.

REFERENCES

- [1] Y. Guan et al., “SSQLi: A Black-Box Adversarial Attack Method for SQL Injection Based on Reinforcement Learning,” *Future Internet*, vol. 15, no. 4, p. 133, 2023.
- [2] S. A. Akhavan et al., “WAFFLED: Exploiting Parsing Discrepancies to Bypass Web Application Firewalls,” in *Proc. Security Research Workshop*, 2025.
- [3] S. A. Akhavan et al., “HTTP-Normalizer: A robust proxy tool for RFC-compliant HTTP validation,” in *Proc. Security Research Workshop*, 2025.
- [4] B. Zhang et al., “BWAFSQLi: Bypassing Web Application Firewall with Adversarial SQL Injections,” *ACM Trans. Softw. Eng. Methodol.*, 2026.
- [5] C. Wu et al., “WAFBOOSTER: Automatic Boosting of WAF Security Against Mutated Malicious Payloads,” in *Proc. Int. Conf. Cybersecurity*, 2025.
- [6] L. Demetrio et al., “WAF-A-MoLE: Evading Web Application Firewalls through Adversarial Machine Learning,” in *Proc. 35th ACM/SIGAPP Symp. Appl. Comput. (SAC)*, 2020.
- [7] L. M. Khan et al., “GSQLi: A GAN-based Approach for Adversarial SQL Injection Sample Generation against WAF,” in *Proc. Int. Conf. Inf. Secur.*, 2024.
- [8] H. O. E. Elebiary et al., “Enhanced Web Payload Classification Using WAMM: An AI-Based Framework for Dataset Refinement and Model Evaluation,” *J. Inf. Secur. Appl.*, 2026.
- [9] OWASP, “Core Rule Set (CRS) Documentation,” OWASP Foundation, 2020.
- [10] M. Author, “A Survey of Web Application Firewall Technologies,” *J. Web Secur.*, 2019.
- [11] T. Author, “Mutation-Based SQL Injection Evasion: An Empirical Study,” in *Proc. Web Secur. Conf.*, 2021.
- [12] J. Author, “Parsing Discrepancies in Web Request Processing: Causes and Security Impact,” *IEEE Access*, 2022.
- [13] K. Author, “Request Smuggling: Attacks and Defenses,” *Comput. Secur.*, 2020.
- [14] R. Author, “Mutation Strategies for SQL Injection Payload Obfuscation,” in *Proc. Int. Symp. Secure Comput.*, 2021.
- [15] R. Fielding et al., “RFC 7230: Hypertext Transfer Protocol (HTTP/1.1): Message Syntax and Routing,” IETF, 2014.
- [16] T. Berners-Lee et al., “RFC 3986: Uniform Resource Identifier (URI): Generic Syntax,” IETF, 2005.